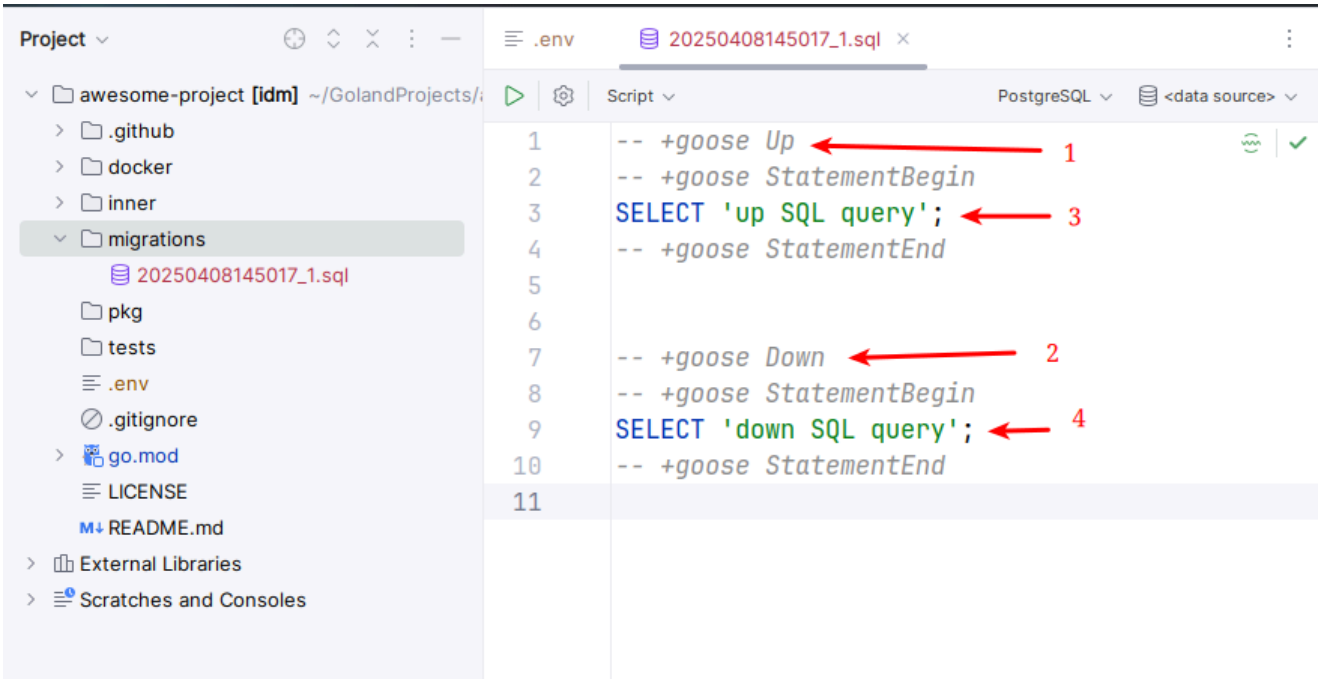
Лекция 9. Мигрируем

В прошлом уроке мы написали DDL скрипты для создания схемы нашей базы данных.
Зачем мы это делали? Почему мы не могли сразу выполнить скрипты в нашей базе данных?
В реальных проектах есть такой термин как "скрипты миграции базы данных".
Почему они называются "скриптами миграции"? Потому что вмете с перемещением вашего приложения в новую среду исполнения (окружение для разработку, окружение для тестов, окужение для промышленной эксплуатации) должна будет переместиться схема базы данных, необходимая для работы вашего приложения.
Есть ошибочное мнение о том, что скрипы миграции нужны для того, чтобы "приложение могло мигрировать на сервер базы данных другого типа". Например, приложение работало с MySQL и лёгким движением руки может быть запущено вместе с PostgreSQL. На самом деле, заменить сервер базы данных одного типа на другой так просто не получится. И от разработчиков потребуется приложить дополнительные усилия для того, чтобы переписать все SQL скрипты с учётом особенностей синтаксиса нового сервера базы данных.
Поэтому скрипты миграции нужны исключительно для того, чтобы перед запуском вашего приложения на пустой базе данных, в этой базе данных была создана схема, с которой сможет работать ваше приложение.
Как вы уже поняли, скрипты миграции не должны запускаться вручную, а должны автоматически выполняться инструментом миграции базы данных пред запуском нашего приложения.
В качестве инструмента миграции могут использоваться:

- функции, написанные вами на языке Go
- различные внешние инструменты: Liquibase, Flyway, goose и многие другие.
  В Go нет промышленного стандарта разработки, обязывающего вас использовать тот или иной инструмент миграции базы данных. В нашем курсе мы подробно рассмотрим один из

популярных инструментов - goose.
Инструкции по установке goose вы можете найти на сайте: https://github.com/pressly/goose

После установки goose, вы можете в терминале, находясь в пакете migrations, создать свой первый скрип миграции:
goose create <script_name> sql
В качестве <script_name> принято указывать номер и/или название задачи, в рамках которой вы выполняете миграцию.
Для нашего урока №9 и задачи "добавить миграцию" полная команда будет выглядеть так:
goose create 9-add-migration sql



Файлы, которые создаёт goose, состоят из двух частей:

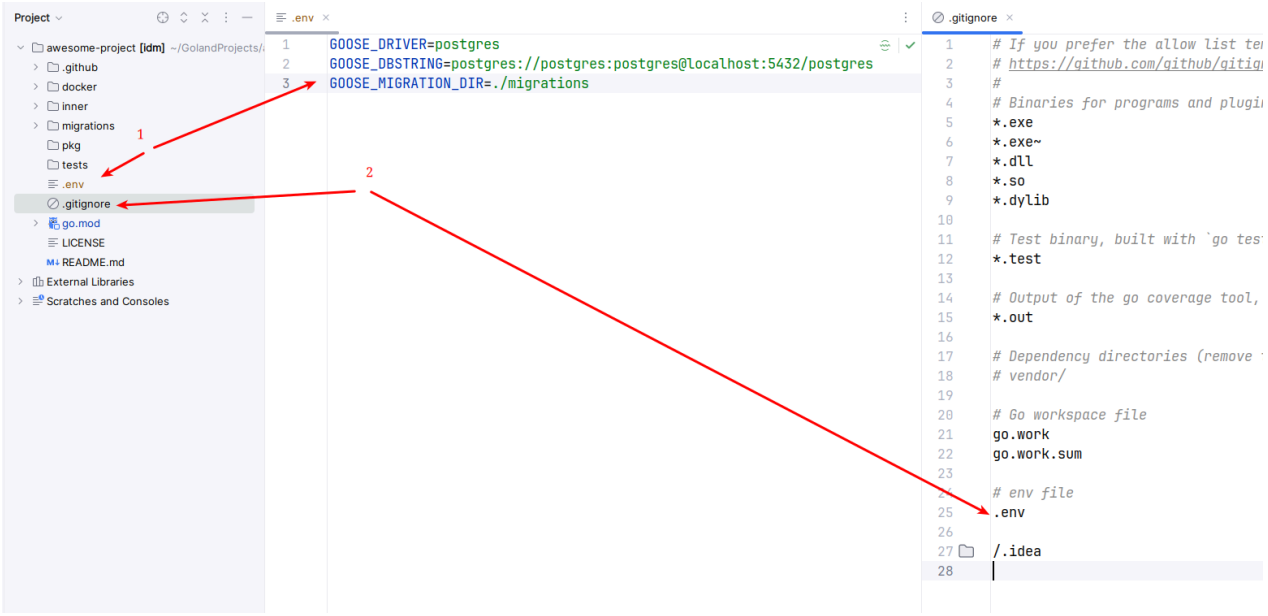
- Up (1)
- Down (2)

  В секции Up необходимо писать скрипты, которые вы хотите применить для создания или изменения схемы вашей базы данных (3).
  В секции Down необходимо писать скрипты (4), которые вы хотите применить, в случае если при выполнении скриптов из секции Up произойдёт ошибка. Эти скрипты должны возвращать базе данных в состояние до начала выполнения скриптов из секции Up.
  После создания первого файла миграции, вам необходимо будет скопировать в его секцию Up скрипты создания таблиц, сделанные вами на прошлом уроке.

В секции Down необходимо разместить сripты, удаляющие созданные вами таблицы.
Следующим шагом, вам необходимо будет передать утилите goose конфигурацию для подключения к вашей базе данных, запущенной в докер контейнере. Эту конфигурацию мы можем передать двумя способами:

- через переменные окружения
- в .env файле

В промышленной разработке принято предавать такого рода конфигурацию в переменных окружения. Вы можете сделать так же. Или для упрощения процесса, можете использовать для этой цели .env файл, который будет располагаться в корне вашего проекта.
Если вы всё сделали по инструкции из прошлого урока, то в вашем .env файле должен получиться следующий конфиг (1):



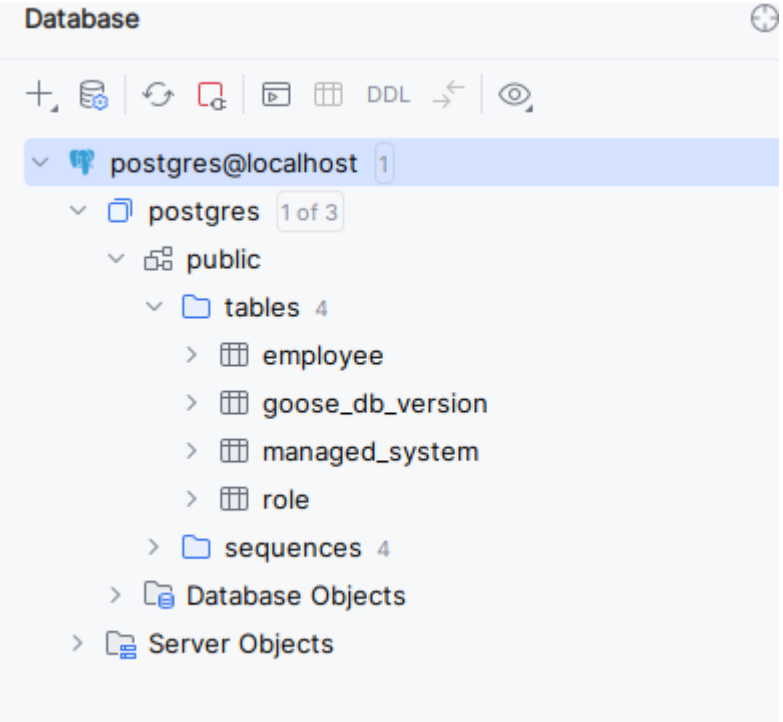
Важно!
В целях безопасности, при использовании .env файла необходимо следить за тем, чтобы этот файл был добавлен в список исключений git в файле .gitignore (2). Если этого не сделать, ваша конфигурация попадёт на GitHub и станет доступной неограниченному кругу лиц, которые могут использовать её во вред вам или компании, в которой вы будете работать.

Запустить миграцию вы можете в терминале, находясь в корне вашего проекта, командой

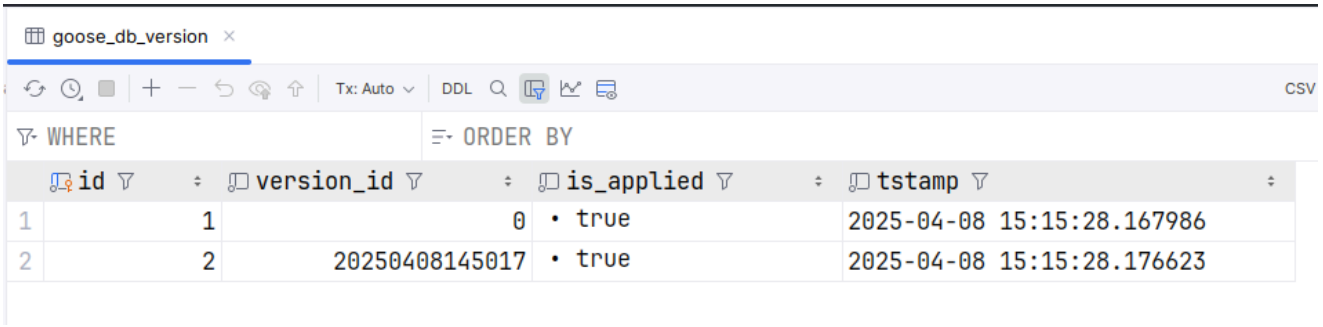
```
goose up
```

```
mikhail@tower:~/GolandProjects/awesome-project$ goose up
2025/04/08 18:15:28 OK   20250408145017_1.sql (8.29ms)
2025/04/08 18:15:28 goose: successfully migrated database to version: 20250408145017
mikhail@tower:~/GolandProjects/awesome-project$
```

После того, как миграция успешно отработает, вы можете в клиенте к вашей базе данных убедиться в том, что все таблички, описанные вами в DDL скриптах, были успешно созданы.



Кроме созданных вами таблиц, goose создаст таблицу goose_db_version, с помощью которой она будет отслеживать, какие срипты миграции уже были выполнены, чтобы не выполнять их повторно.



	id	version_id	is_applied	tstamp
1	1	0	true	2025-04-08 15:15:28.167986
2	2	20250408145017	true	2025-04-08 15:15:28.176623

Задание на сегодня:
Выполнить шаги по созданию схемы базы данных, описанные в уроке.